Lecture 11
copy constructor

Today's Lecture Contents

- Copy constructor

Constructors and Copy Constructors

- All class definitions include a default constructor.
- The ***default constructor*** does not take any arguments.
- It is used to initialize object data fields when no other arguments are specified.
- Besides a default constructor, a class definition also provides a default ***copy constructor***
- A **copy constructor** is a special constructor that is called **whenever a new object is created and initialized with another object's data.**
- For example:

```
Rectangle r1(1,18); //object r1 with length and width members;
Ractangle r2(r1);
```

- The above statement creates object r2 and initializes it with the r1's data.
- It uses the default copy constructor provided by the compiler to make a duplicate copy (r2) of the already existing object (r1).

Example: Default copy constructor

```
class Rectangle {
public:
Rectangle(int w=5,int l=10) {
width=w; length=l;}
Void printDimensions() {
Cout<<"Rectangle Width= "<<width<<" length= "<<length<<endl; }
private:
  int width;
  int length;
};
int main() {
Rectangle  r1(10,40);
Rectangle  r2(r1); //creates and initializes r2 by calling default
copy constructor
r1.printDimensions();
r2.printDimensions();
}
```

```
Rectangle width = 10 and lenth = 40

Rectangle width = 10 and lenth = 40
```

Non-default Copy Constructor

- Like a normal constructor it is also possible to explicitly provide a copy constructor.
- In the presence of the user-provided copy constructor, the default copy constructor will not be called.
- Most of the time, the default copy constructor is sufficient, but there will be situations where we will need an explicit copy constructor.

- Signature of the copy constructor may look something like this:

```
Rectangle::Rectangle(const Rectangle &obj);
```

- And its full implementation may look something like this:

```
Rectangle::Rectangle(const Rectangle &obj) {
width = obj.width;
length = obj.length;
}
```

Example: non-Default copy constructor

```
class Rectangle {
public:
Rectangle(int w=5,int l=10) {
width=w; length=l;}
Rectangle(const Rectangle &obj) {
width = obj.width;     length = obj.length;
cout<<"Rectangle created using non-default copy constructor: ";}
Void printDimensions() {
Cout<<"Rectangle Width= "<<width<<" length= "<<length<<endl; }
private:
  int width;
  int length;
};
int main() {
Rectangle  r1(10,40);
Rectangle  r2(r1); //creates and initializes r2 by calling copy
constructor
r1.printDimensions();
r2.printDimensions();
}
```

```
Rectangle created using non-default copy constructor:
                Rectangle width = 10 and lenth = 40

                Rectangle width = 10 and lenth = 40
```

when is copy constructor called?

- When is a copy of an object made?
  - Passing object by value as a parameter.

```
void displayRectangle(Rectangle R);
```

  - returning an object from a function by value.

```
Rectangle createRectangle();
```

  - Constructing one object based on another of the same class.

```
Rectangle r1;
Rectangle r2(r1);
```

Shallow copy

- The default copy constructor simply copies the bytes of the object.
  - This is called a **shallow copy**.
- As long as all of your object's data is inside the object, a **shallow copy** is fine.
  - For example, we used default copy constructor to make copy of our **Rectangle class object**.
- However, if an *object has a pointer to dynamic storage* as a data member, not all of its data is inside the object.
- The dynamic storage is separate from the object itself, and resides on the free store.
- A shallow copy will simply copy the pointer to this dynamic storage without actually making a copy of the storage contents.
- The result is that you end up with two different objects pointing to the same chunk of dynamic storage.

Example: Shallow copy

```
class dynamicvar {
private:
int * ptr;
public:
dynamicvar(int n) {
cout<<"in constructor: Allocating variable on the heap: "<<endl;
ptr = new int;
*ptr = n;
 }
~dynamicvar(){
cout<<"In Destructor: Dynamically allocated variable is being deleted:";
delete ptr; }
};
int main() {
dynamicvar D1(5);
D1.display();
        {
        Dynamicvar D2(D1);//calling default copy constructor
        D2.modify(10);
        D1.display(); //what will it print?
        }
        D1.display()// //what will it print?
}
```

Example: Shallow copy

- Write a customized copy constructor for the class on the previous slide.

```
class dynamicvar {
private:
int * ptr;
public:
dynamicvar(int n) {
cout<<"in constructor: Allocating variable on the heap: "<<endl;
ptr = new int;
*ptr = n;
 }
~dynamicvar(){
cout<<"In Destructor: Dynamically allocated variable is being deleted:";
delete ptr; }
};
```

User defined copy constructor

- The below user-defined copy constructor does ***deep copy***, instead of a shallow copy.

```
class dynamicvar {
public:
dynamicvar(int n) {
cout<<"in constructor: Allocating variable on the heap: "<<endl;
ptr = new int;
*ptr = n;
 }
dynamicvar(const dynamicvar & obj) {
        cout<<"in copy constructor: copying object: "<<endl;
        ptr = new int;
        *ptr = *(obj.ptr);
 }
~dynamicvar(){
cout<<"In Destructor: Dynamically allocated variable is being deleted:";
delete ptr;  }
};
```

Shallow Copy: Another Example

- Default copy constructors work fine unless the class contains pointer data members.
- For example, If name had a dynamically allocated array of characters (i.e., one of the data members is a pointer to a char).

```
 #include <string.h>
class name {
  public:
name(const char* name_ptr = "");          //default constructor
   ~name() { delete ptr; }              //destructor

private:
    char* ptr;  //pointer to name
    int length; //length of name including null char
};

name::name(const char* name_ptr) {    //constructor
  length = strlen(name_ptr);    //get name length
  ptr = new char[length+1];     //dynamically allocate
  strcpy(ptr, name_ptr);        //copy name into new space
}
```

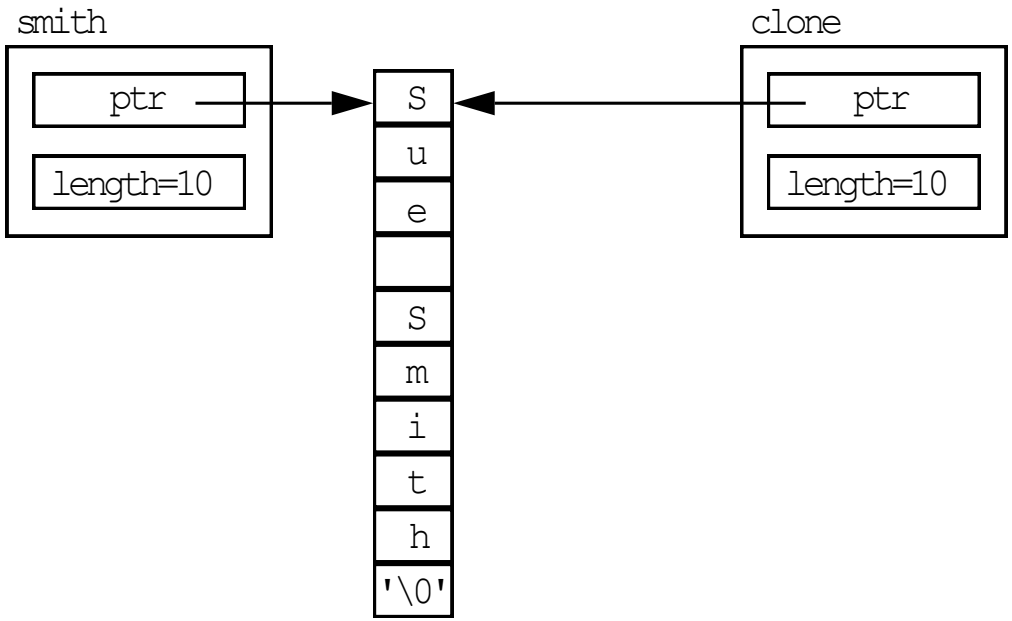
```
int main() {
name smith("Sue Smith"); //one argument constructor
name clone(smith);//using default copy constructor

}
```

Copy Constructors

- the following shallow copy is disastrous!

```
name smith("Sue Smith"); //one arg constructor used
name clone(smith);       //default copy constructor used
```



Defining your own Copy Constructors

- To resolve the issue, we must write a copy constructor whenever dynamic member is allocated on an object-by-object basis.→ This copy constructor makes a "deep copy" of the object

- A copy constructor takes an instance of the same class as a constant reference argument.

- They have the form:

```
name(const name & class_object);
```

- Notice the name of the “function” is the same name as the class, and has no return type
- The argument’s data type is that of the class, passed as a constant reference

Defining your own Copy Constructors

```
class name {

char* ptr;  //pointer to name
    int length; //length of name including nul char
public:
    name(const char* = "");          //default constructor
    name(const name &);       //copy constructor
    ~name() { delete ptr};                    //destructor
};

name::name(const char* name_ptr) {   //constructor
  length = strlen(name_ptr);    //get name length
  ptr = new char[length+1];     //dynamically allocate
  strcpy(ptr, name_ptr);        //copy name into new space
}

name::name(const name &obj) {  //copy constructor
  length = obj.length;          //get length
  ptr = new char[length+1];     //dynamically allocate
  strcpy(ptr, obj.ptr);         //copy name into new space
}
```

Defining your own Copy Constructors

- Now, when we use the following constructors for initialization, the two objects no longer share memory but have their own allocated

```
Student smith("Sue Smith"); //one argument constructor
Student clone(smith);//User-defined copy constructor used
```